Informe del proyecto integrador

Grupo 3 · Personal

Unidad II · Desarrollo de aplicaciones y plataformas · 6 de noviembre de 2026

1. Objetivo y alcance

Desarrollar el módulo de Personal de una residencia de adultos mayores: consultar indicadores, listar con filtros, mantener trabajadores y transferir su información a SQL Server. Actividades, incidentes y residentes se leen únicamente como apoyo para los cálculos; no se implementan sus módulos de mantenimiento.

Arquitectura y herramientas

AdminLTE 4.10.1 y Bootstrap 5.3.8 forman la interfaz. JavaScript usa fetch para consultar la API Flask 3.1.3. PyMySQL conecta al origen MySQL Community Server 8.4.13 y pyodbc al destino SQL Server. Chart.js 4.5.1 construye las gráficas y Tabulator 6.6.1 presenta el listado. Las credenciales se leen desde .env.

Indicadores exigidos y fórmulas SQL

Indicador	Cálculo	Referencia del script
Personal activo	COUNT(*) con estado ACTIVO	15
Distribución por cargo	COUNT(*) agrupado por cargo	9 cargos: Enfermera 4, Cuidador 3, Fisioterapeuta 2
Antigüedad media	AVG(TIMESTAMPDIFF(MONTH, fecha_ingreso, CURDATE())) / 12	Variable según la fecha
Residentes por cuidador	Residentes activos / personal con cargo Cuidador	20 / 3 = 6,7
Actividades por responsable	Actividades / responsables distintos	12 / 4 = 3
Incidentes por reportante	Incidentes con reportante / reportantes distintos	12 / 4 = 3

NULLIF y COALESCE evitan dividir entre cero y devuelven 0. El periodo selecciona personal por fecha_ingreso; la carga incluye sus actividades e incidentes asociados.

Alcance de la verificación

Para reproducir las referencias del Word se prepararon bases de ensayo separadas: sistema_residencia_pruebas y residencia_sqlserver_pruebas. La instancia temporal usó el puerto 5001. La aplicación normal continúa en 5000. No se restauraron los datos operativos para forzar coincidencias con los valores de clase.

2. Dashboard y gráfica comparativa

Las seis tarjetas reciben cálculos de la API. La captura de ensayo muestra 15 activos, 9 cargos, antigüedad media de 2,3 años, 6,7 residentes por cuidador y medias de 3 actividades y 3 incidentes por persona responsable.

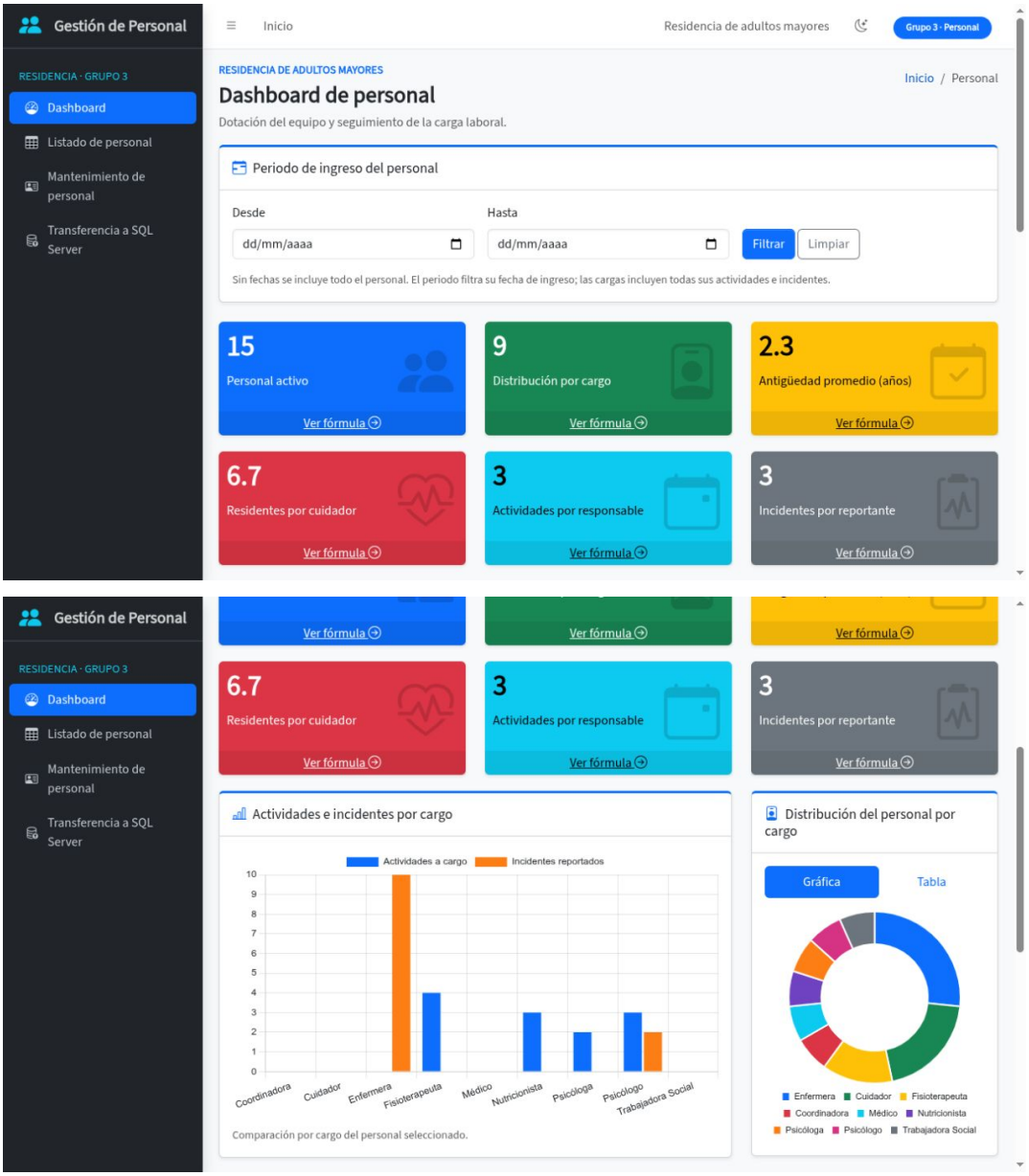


Figura 1. Dashboard y gráficas con los datos originales en la instancia de ensayo. Azul representa actividades y naranja incidentes. La gráfica circular y su tabla muestran la distribución de personas por cargo.

Referencias verificadas: Enfermera 0 actividades / 10 incidentes; Fisioterapeuta 4 / 0; Nutricionista 3 / 0; Psicólogo 3 / 2; Psicóloga 2 / 0. Las consultas preagrupan actividades e incidentes por separado antes de unirlos, evitando multiplicar los conteos.

3. Listado con filtros y mantenimiento CRUD

El listado combina cargo, estado, especialidad, fechas inclusivas de ingreso y búsqueda por nombre, apellido o código. Las condiciones se unen con AND y los filtros vacíos se omiten. Enfermera + ACTIVO devuelve cuatro registros; el rango de ingreso de todo 2025 devuelve seis en los datos de clase.

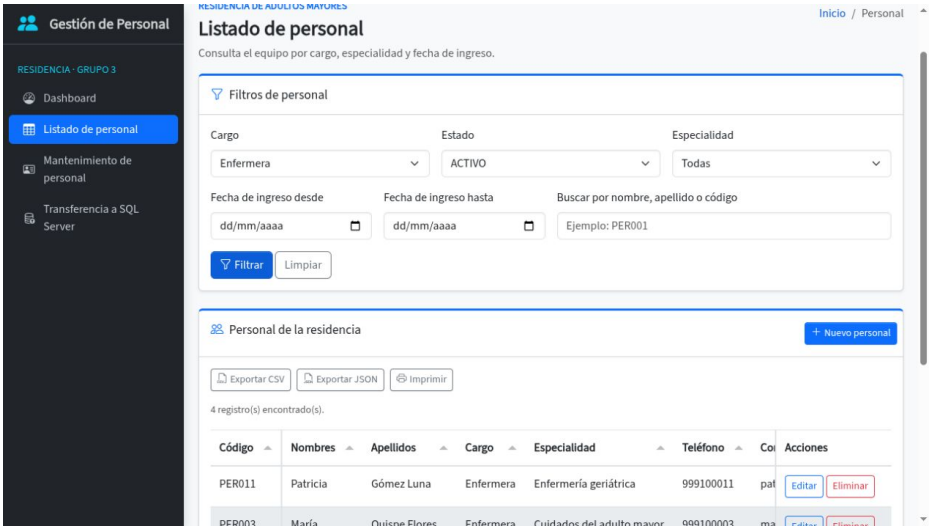


Figura 2. Filtro combinado comprobado desde el navegador.

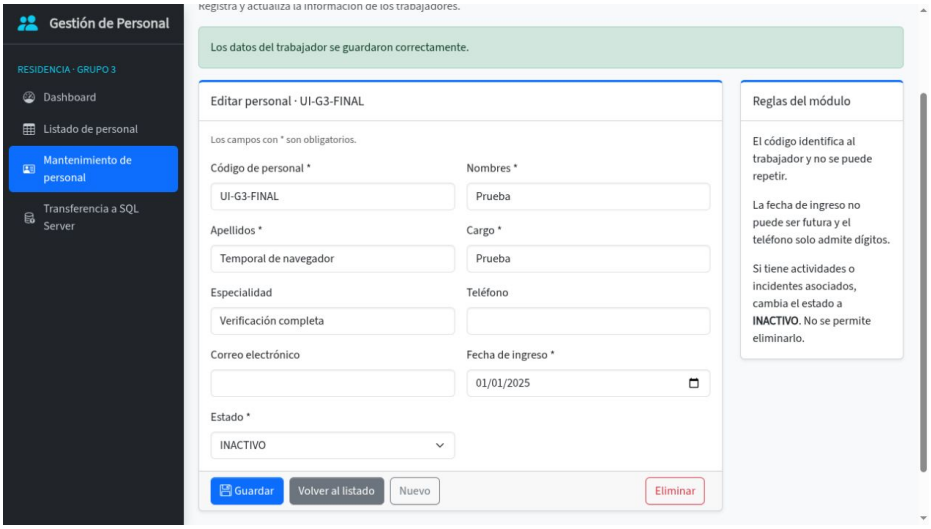


Figura 3. Registro temporal creado y editado a INACTIVO desde la interfaz, después eliminado.

POST crea (201), GET consulta (200), PUT actualiza (200) y DELETE elimina (204 sin cuerpo). Se exige código único, nombres, apellidos, cargo, fecha no futura y estado válido. Se validan longitudes, correo y teléfono de solo dígitos. Un duplicado devuelve 409 y un dato inválido 400.

Regla de negocio: si tiene actividades o incidentes, la API bloquea el borrado con 409 y sugiere INACTIVO. Se comprobó el mensaje visible para PER001 sin eliminarlo. La captura adicional se entrega como evidencias/regla_409.png.

4. Transferencia y verificación en SQL Server

El botón pide confirmación y llama a POST /api/personal/transferir. Copia toda la tabla, sin aplicar los filtros de la pantalla. MERGE busca por id_personal: inserta si falta y actualiza si existe. El destino conserva los IDs sin IDENTITY, no tiene llaves foráneas y añade fecha_transferencia.

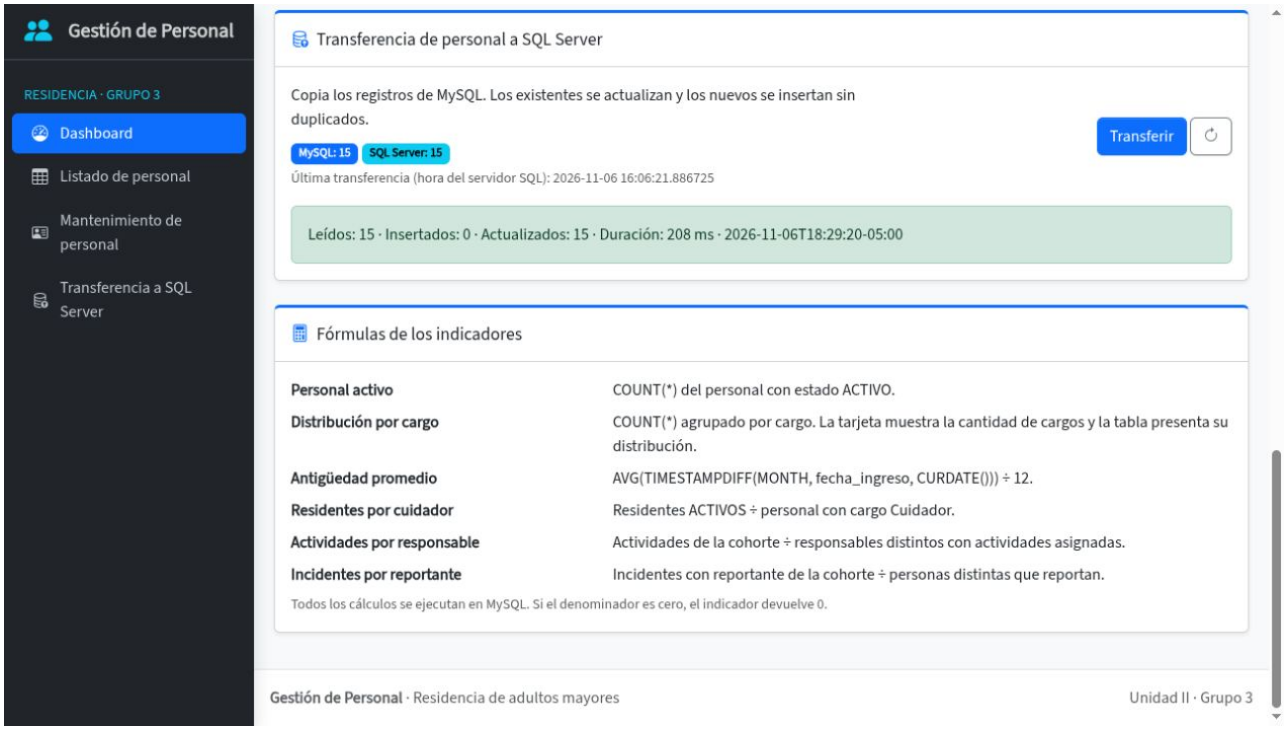


Figura 4. Transferencia realizada desde el navegador sobre las bases de ensayo: 15 leídos, 0 insertados, 15 actualizados y conteos 15/15 al repetir.

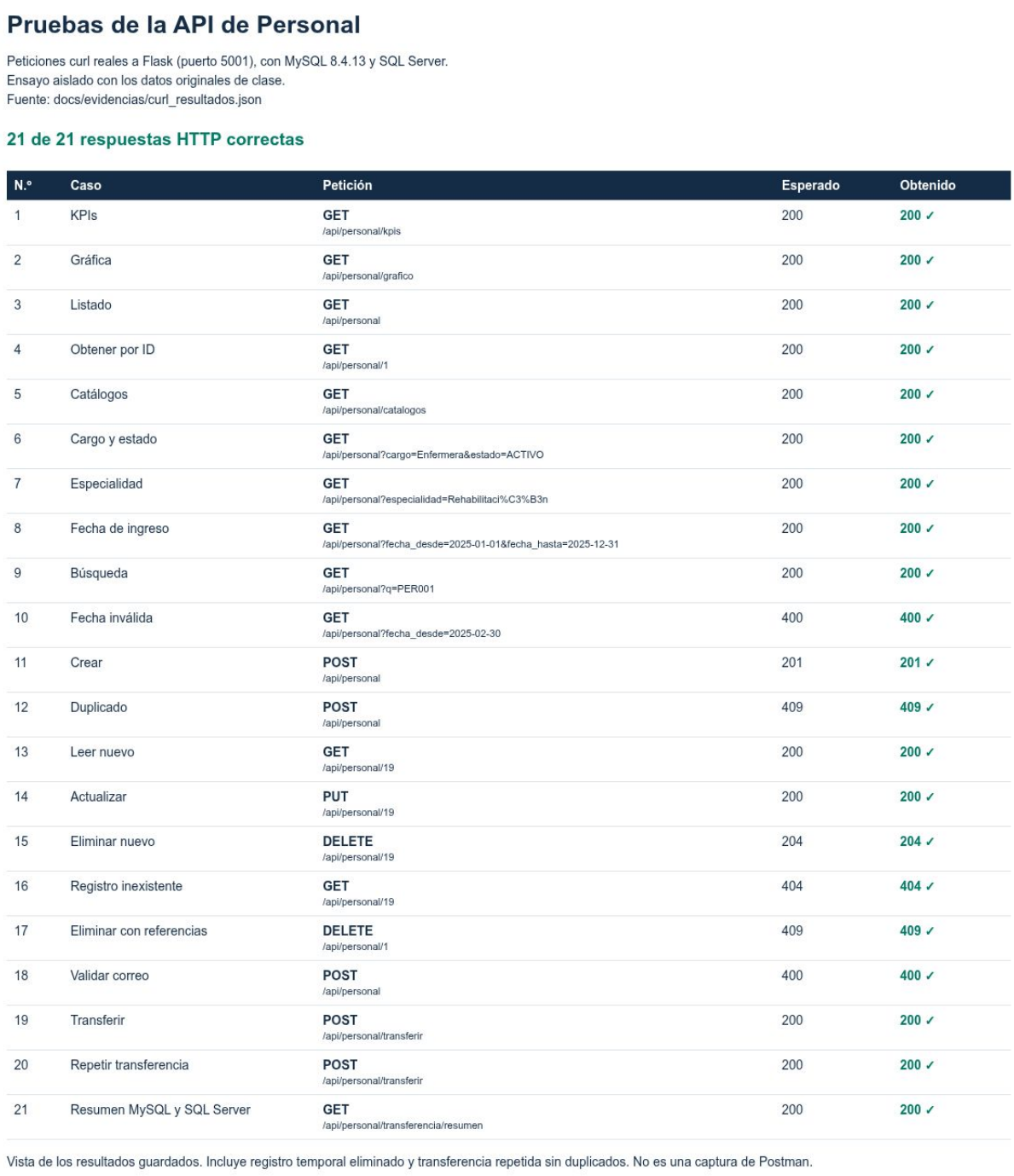
Comprobación	Resultado
Transacción de destino	commit si termina; rollback si falla. Fallo de escritura simulado comprobado.
Comparación directa	Todas las filas actuales del ensayo coinciden campo por campo en ambos motores.
Estructura real	Llave primaria presente; cero columnas IDENTITY y cero llaves foráneas.
Trazabilidad	Todas las filas tienen fecha_transferencia. Se muestra la hora del servidor SQL.
Límite del proceso	La copia no sincroniza borrados. Puede conservar filas históricas solo en el destino.

En la aplicación operativa se observó este último caso durante la revisión. Se conservaron esas filas. El informe distingue ese estado cambiante del ensayo reproducible con los datos originales.

```
SELECT id_personal, codigo_personal, estado, fecha_transferencia
FROM residencia_sqlserver_pruebas.dbo.personal
ORDER BY id_personal;
```

5. Evidencia de pruebas de la API con curl

Se ejecutaron 21 peticiones curl reales contra la instancia aislada y sus dos motores. Se comprobaron consultas, filtros, CRUD, duplicado, correo y fecha inválidos, inexistente, referencias y transferencia repetida. Los resultados completos se entregan en curl_resultados.json y curl_resultados.txt.



Pruebas de la API de Personal

Peticiones curl reales a Flask (puerto 5001), con MySQL 8.4.13 y SQL Server.
Ensayo aislado con los datos originales de clase.
Fuente: docs/evidencias/curl_resultados.json

21 de 21 respuestas HTTP correctas

N.º	Caso	Petición	Esperado	Obtenido
1	KPIs	**GET** /api/personal/kpis	200	200 ✓
2	Gráfica	**GET** /api/personal/grafico	200	200 ✓
3	Listado	**GET** /api/personal	200	200 ✓
4	Obtener por ID	**GET** /api/personal/1	200	200 ✓
5	Catálogos	**GET** /api/personal/catalogos	200	200 ✓
6	Cargo y estado	**GET** /api/personal?cargo=Enfermera&estado=ACTIVO	200	200 ✓
7	Especialidad	**GET** /api/personal?especialidad=Rehabilitaci%C3%B3n	200	200 ✓
8	Fecha de ingreso	**GET** /api/personal?fecha_desde=2025-01-01&fecha_hasta=2025-12-31	200	200 ✓
9	Búsqueda	**GET** /api/personal?q=PER001	200	200 ✓
10	Fecha inválida	**GET** /api/personal?fecha_desde=2025-02-30	400	400 ✓
11	Crear	**POST** /api/personal	201	201 ✓
12	Duplicado	**POST** /api/personal	409	409 ✓
13	Leer nuevo	**GET** /api/personal/19	200	200 ✓
14	Actualizar	**PUT** /api/personal/19	200	200 ✓
15	Eliminar nuevo	**DELETE** /api/personal/19	204	204 ✓
16	Registro inexistente	**GET** /api/personal/19	404	404 ✓
17	Eliminar con referencias	**DELETE** /api/personal/1	409	409 ✓
18	Validar correo	**POST** /api/personal	400	400 ✓
19	Transferir	**POST** /api/personal/transferir	200	200 ✓
20	Repetir transferencia	**POST** /api/personal/transferir	200	200 ✓
21	Resumen MySQL y SQL Server	**GET** /api/personal/transferencia/resumen	200	200 ✓

Vista de los resultados guardados. Incluye registro temporal eliminado y transferencia repetida sin duplicados. No es una captura de Postman.

Figura 5. Captura de la vista de resultados guardados de curl: 21/21 códigos HTTP esperados. Es una visualización de peticiones reales; no se presenta como una ejecución de Postman. La colección Postman se incluye para importarla.

6. Cobertura, estructura y conclusión

Verificación	Resultado y alcance
Suite automatizada	19/19 pruebas: KPIs, gráfica, filtros, CRUD, reglas, JSON, transferencia y portabilidad.
Revisión adicional	104/104 comprobaciones: límites de campos, tipos, obligatorios, JSON inválido, códigos HTTP, recursos, datos y esquema del destino.
Navegador	Filtros, crear/editar/eliminar temporal, bloqueo por referencias, gráficas/tabla y confirmación de transferencia.
Fallos controlados	Base origen no disponible (configuración de ensayo), destino no disponible y rollback de carga simulados.
Conservación	El temporal del navegador se eliminó. Las comprobaciones adicionales verificaron que no alteraron los registros existentes del ensayo.
Límite de portabilidad	Ejecución real en Linux. Rutas y drivers de Windows simulados; pendiente prueba en un equipo Windows real.

Estructura de entrega

```
grupo3_personal/
  app.py, config.py, db_mysql.py, db_sqlserver.py
  modulo/routes.py, modulo/transferencia.py
  sql/sqlserver_personal.sql
  static/
  html/index3.html, html/tables/data.html, html/crud.html
  postman/coleccion.json
  .env.example, requirements.txt
```

Se conservan los nombres requeridos. Los comentarios en español explican las secciones. El código usa consultas parametrizadas, usuarios de aplicación y credenciales privadas excluidas del ZIP. Los archivos de validación, plantilla común y lanzadores son complementarios.

Ejecución y conclusión

MySQL 8 utiliza el puerto 3307 en este Linux y SQL Server el 1433; cada equipo configura sus valores. Se inicia con iniciar.bat en Windows o bash iniciar.sh en Linux. No se necesita XAMPP. El lanzador no importa bases ni administra servicios.

El módulo implementa el alcance del Grupo 3: indicadores y gráfica, listado filtrado, mantenimiento validado y transferencia transaccional sin duplicados por ID. La evidencia cubre los recorridos y reglas requeridos. El Word menciona SSMS; las comprobaciones directas se realizaron contra SQL Server y el alumno utiliza DataGrip.

Fuentes: Plan de trabajo Unidad II, secciones 3, 7 y 12; SQL y plantilla proporcionados; código del proyecto. Evidencias: pruebas_mysql8.txt, verificacion_completa.json, verificacion_navegador.json y curl_resultados.json.